

(1) (d) Use the Jeans Theorem to compute the velocity disp. $\sigma^2(r)$. Use symbolic notation.

Podemos comenzar usando el potencial de Plummer visto en clases:

$$\rho(r) = \frac{3M}{4\pi b^3 (1+r^2/b^2)^{5/2}}, \text{ y la masa encerrada corresponde a}$$

$$\hookrightarrow M_{enc}(r) = 4\pi \int_0^r \rho(r') r'^2 dr'$$

$$\therefore M_{enc} = 4\pi \int_0^r \frac{3M}{4\pi b^3 (1+r'^2/b^2)^{5/2}} r'^2 dr' = \frac{3M}{b^3} \int_0^r b \cdot \frac{r'^2}{(1+r'^2/b^2)^{5/2}} dr' = 3M b^2 \int_0^r \frac{r'^2}{(1+r'^2/b^2)^{5/2}} dr'$$

$$M_{enc} = 3M b^2 \left(\frac{r^3}{3b^2 (b^2+r^2)^{3/2}} \right) = \frac{Mr^3}{(b^2+r^2)^{3/2}} //$$

De clases sabemos que $\rightarrow \sigma_r^2(r) = \langle v_r^2 \rangle = \frac{1}{v(r)} \int_r^\infty \frac{G M_{enc}(r') v(r')}{r'^2} dr'$ con $v(r)$ es dens. de estrellas
(a adl podemos reemplazar directamente por la densidad $\rho(r)$)

$$\begin{aligned} \hookrightarrow \sigma_r^2(r) &= \frac{1}{\rho(r)} \int_r^\infty \frac{G M_{enc}(r') \rho(r')}{r'^2} dr' = \frac{4\pi b^3 (1+r^2/b^2)^{5/2}}{3M} \cdot \int_r^\infty \frac{G}{r'^2} \left(\frac{Mr'^3}{(b^2+r'^2)^{3/2}} \right) \frac{3M}{4\pi b^3 (1+r'^2/b^2)^{5/2}} dr' \\ &= GM (1+r^2/b^2)^{5/2} \int_r^\infty \frac{r'}{(b^2+r'^2)^{3/2}} \cdot \frac{b^5}{(b^2+r'^2)^{3/2}} dr' = GM b^5 (1+r^2/b^2)^{5/2} \int_r^\infty \frac{r'}{(b^2+r'^2)^4} dr' \end{aligned}$$

Haciendo el cambio de variable a combinación $\rightarrow x = r'^2 + b^2 \quad \therefore dx = 2r' dr'$
Entonces

$$\hookrightarrow \int_r^\infty \frac{r'}{(r'^2+b^2)^4} dr' = \int_r^\infty \frac{dx}{2x^4} = \frac{1}{2} \int_r^\infty \frac{1}{x^4} = \frac{1}{2} \left(\frac{-1}{3(x)^3} \right) = \frac{1}{6} (r^2+b^2)^{-3} //$$

$$\text{Entonces considerando la densidad esférica } \sigma^2(r) = 3 \cdot \sigma_r^2(r) \quad \therefore \sigma^2(r) = 3 \cdot \frac{GM}{6(r^2+b^2)^{1/2}} = \frac{GM}{2(r^2+b^2)^{1/2}} //$$

(2) Asumiendo un modelo para el disco estelar de la MW como;

$$\phi_{MW}(R, z) = \frac{-GM}{(R^2 + (a + \sqrt{z^2 + b^2})^2)^{1/2}} \quad \text{con } M = 6.0 \times 10^{10} M_\odot, \quad a = 3 \text{ kpc}, \quad b = 280 \text{ pc}, \quad R_g = 8 \text{ kpc}$$

(a) Solve for the motion of the guiding center and compute $R(R_g)$, L_z and the circular velocity V_{circ} in km/s.

De clase sabemos que $V^2(R) = \frac{1}{R} \left(\frac{\partial \phi}{\partial R} \right)_{(R,0)}$, donde ϕ denota el potencial dado. Por lo tanto, denotando b como puede a z.

$$\rightarrow \frac{\partial \phi_{MW}}{\partial R} = \frac{\partial}{\partial R} \left(\frac{-GM}{(R^2 + (a + \sqrt{z^2 + b^2})^2)^{1/2}} \right) = \frac{\partial}{\partial R} \left(\frac{-GM}{\sqrt{R^2 + (x)^2}} \right) = GM \cdot \frac{1}{\sqrt{R^2 + (x)^2}} \cdot \frac{2R}{2} \cdot \frac{1}{R^2 + x^2}$$

$$\frac{\partial \phi_{MW}}{\partial R} = \frac{GMR}{(R^2 + (a + \sqrt{z^2 + b^2})^2)^{3/2}}, \text{ reemplazando } R = R_g \text{ y } z = 0 \rightarrow \frac{\partial \phi_{MW}}{\partial R} = \frac{GMR_g}{(2R_g^2 + (a+b)^2)^{3/2}}$$

Ahora reemplazando $\rightarrow \Omega^2(R_g) = \frac{1}{R_g} \left(\frac{\partial \phi}{\partial R} \right)_{R,0} \rightarrow \Omega(R_g) = \left(\frac{GM}{(R_g^2 + (a+b)^2)^{3/2}} \right)^{1/2}$

Los valores dados

$\rightarrow R_g^2 + (a+b)^2 = (8000^2 + (3000+200)^2)^{3/2}$ y pasamos a metros $\rightarrow 1.9 \times 10^{11} \text{ [m}^3]$

$\Omega(R_g) = \left(\frac{GM}{r} \right)^{1/2} = \left(\frac{6.67 \times 10^{-11} \text{ m}^3/\text{kg s}^2 \cdot (6.8 \times 10^{30} \text{ kg})}{1.9 \times 10^{11} \text{ m}^3} \right)^{1/2} = 1.35 \times 10^{14} \text{ kg} = \left(4.739 \times 10^{-21} \frac{1}{\text{s}^2} \right)^{1/2}$

$\Omega(R_g) = 6.54 \times 10^{-16} \text{ 1/s}$

luego para obtener L_z usamos la relación vista en clases $\rightarrow \Omega^2(R_g) = \frac{L_z^2}{R_g^4} \therefore L_z = R_g^2 \Omega(R_g) = (2.47 \cdot 10^{20})^2 \cdot 6.54 \cdot 10^{-16} \frac{\text{m}^2}{\text{s}}$

$\therefore L_z = 3.987 \frac{\text{kg m}^2}{\text{s}}$ y la velocidad angular $\rightarrow v_{\text{circ}}(R_g) = R_g \Omega(R_g) = (2.47 \cdot 10^{20} \text{ m}) \cdot (6.54 \times 10^{-16} \frac{1}{\text{s}})$

$v_{\text{circ}} = 161.53 \frac{\text{km}}{\text{s}}$

(b) Find the epicyclic freq. κ and vertical frequency ν at R_g . What is the ratio κ/Ω ?

De clases usamos tenemos que $\rightarrow \kappa^2(R_g) = \left(2 \frac{d\Omega^2}{dR} + 4\Omega^2 \right)_{R_g}$ y $\nu^2(R_g) = \left(\frac{\partial^2 \phi}{\partial z^2} \right)_{(R_g,0)}$

Siendo simétrico, esto lo hice con Mathematica

$\rightarrow \nu^2(R_g) = \frac{GM(a+b)}{b((a+b)^2 + R_g^2)^{3/2}}$ y reemplazando numéricamente $\nu(R_g) = 2.36 \cdot 10^{-15} \left[\frac{1}{\text{s}} \right]$

$\kappa^2(R_g) = \left(R \cdot \frac{d}{dR} \left(\frac{GM}{(R^2 + (a+b)^2)^{3/2}} \right) + \frac{4GM}{(R^2 + (a+b)^2)^{3/2}} \right)_{R_g}$
 $= \left(\frac{-3GM R^2}{(R^2 + (a+b)^2)^{5/2}} + \frac{4GM}{(R^2 + (a+b)^2)^{3/2}} \right)_{R_g} = \frac{GM}{(R_g^2 + (a+b)^2)^{3/2}} \cdot \left(\frac{-3R_g^2}{R_g^2 + (a+b)^2} + 4 \right)$

Reemplazando con los valores numéricos encontramos $\rightarrow \kappa(R_g) = 8.25 \times 10^{-16} \left[\frac{1}{\text{s}} \right]$

Así finalmente la razón $\frac{\kappa(R_g)}{\Omega(R_g)} \approx 1.2$

```
import numpy as np
import matplotlib.pyplot as plt
import astropy.units as u
import gala.potential as gp
import gala.dynamics as gd
from gala.units import UnitSystem
import astropy.constants as c
```

Pregunta 1(a)

Ahora ocupamos las de masa y B de omega centuari dadas en clases

```
pip install gala
```

```
Collecting gala
```

```
  Downloading gala-1.7.1-cp310-cp310-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (12.6 MB)
----- 12.6/12.6 MB 19.1 MB/s eta
```

```
0:00:00
```

```
Requirement already satisfied: numpy>=1.20 in /usr/local/lib/python3.10/dist-
packages (from gala) (1.23.5)
```

```
Requirement already satisfied: scipy>=1.8 in
/usr/local/lib/python3.10/dist-packages (from gala) (1.11.4)
```

```
Requirement already satisfied: astropy>=5.0 in
/usr/local/lib/python3.10/dist-packages (from gala) (5.3.4)
```

```
Requirement already satisfied: pyyaml in
/usr/local/lib/python3.10/dist-packages (from gala) (6.0.1)
```

```
Requirement already satisfied: cython>=0.29 in
/usr/local/lib/python3.10/dist-packages (from gala) (3.0.6)
```

```
Requirement already satisfied: pyerfa>=2.0 in
/usr/local/lib/python3.10/dist-packages (from astropy>=5.0->gala)
(2.0.1.1)
```

```
Requirement already satisfied: packaging>=19.0 in
/usr/local/lib/python3.10/dist-packages (from astropy>=5.0->gala)
(23.2)
```

```
Installing collected packages: gala
```

```
Successfully installed gala-1.7.1
```

```
unidades = UnitSystem(u.pc, u.Myr, u.radian, u.Msun)
```

```
mass = 4.05e6
```

```
b = 26
```

```
plummer = gp.PlummerPotential(m=mass, b=b, units=unidades)
```

```
r_init = 10
```

```
v_circ = plummer.circular_velocity([r_init, 0, 0]).value
```

```
w0 = gd.PhaseSpacePosition(pos=[r_init, 0, 0], vel=[0, v_circ, 0])
```

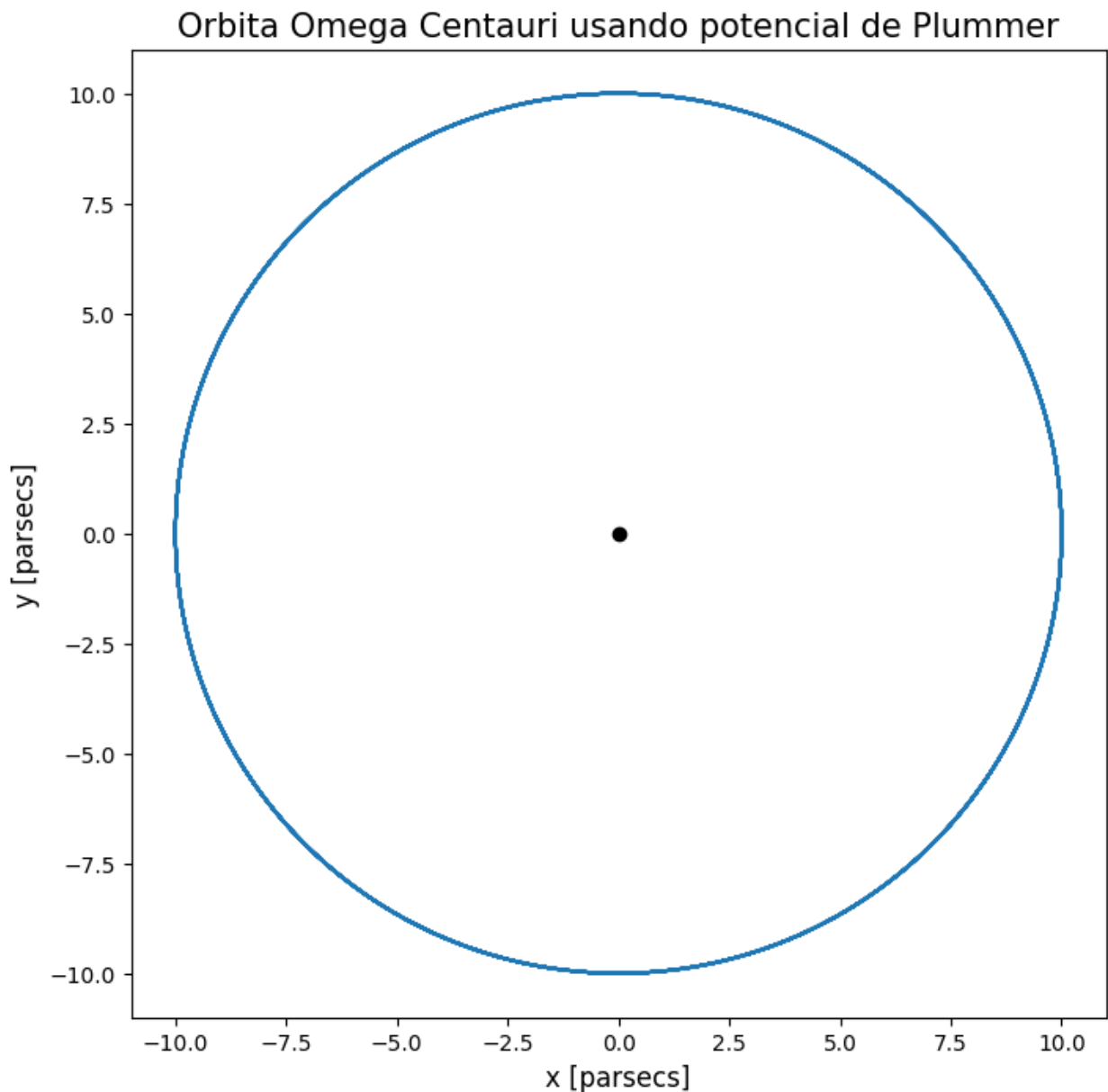
```
orbital_period = 2 * np.pi * r_init / v_circ
```

```

orbit = plummer.integrate_orbit(w0, dt=0.001, n_steps=int(10 *
orbital_period / 0.001))

fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(0, 0, color='black', marker='o')
ax.plot(orbit.x, orbit.y)
ax.set_xlabel('x [parsecs]', size=12)
ax.set_ylabel('y [parsecs]', size=12)
ax.set_title('Orbita Omega Centauri usando potencial de Plummer',
size=15)
plt.show()

```



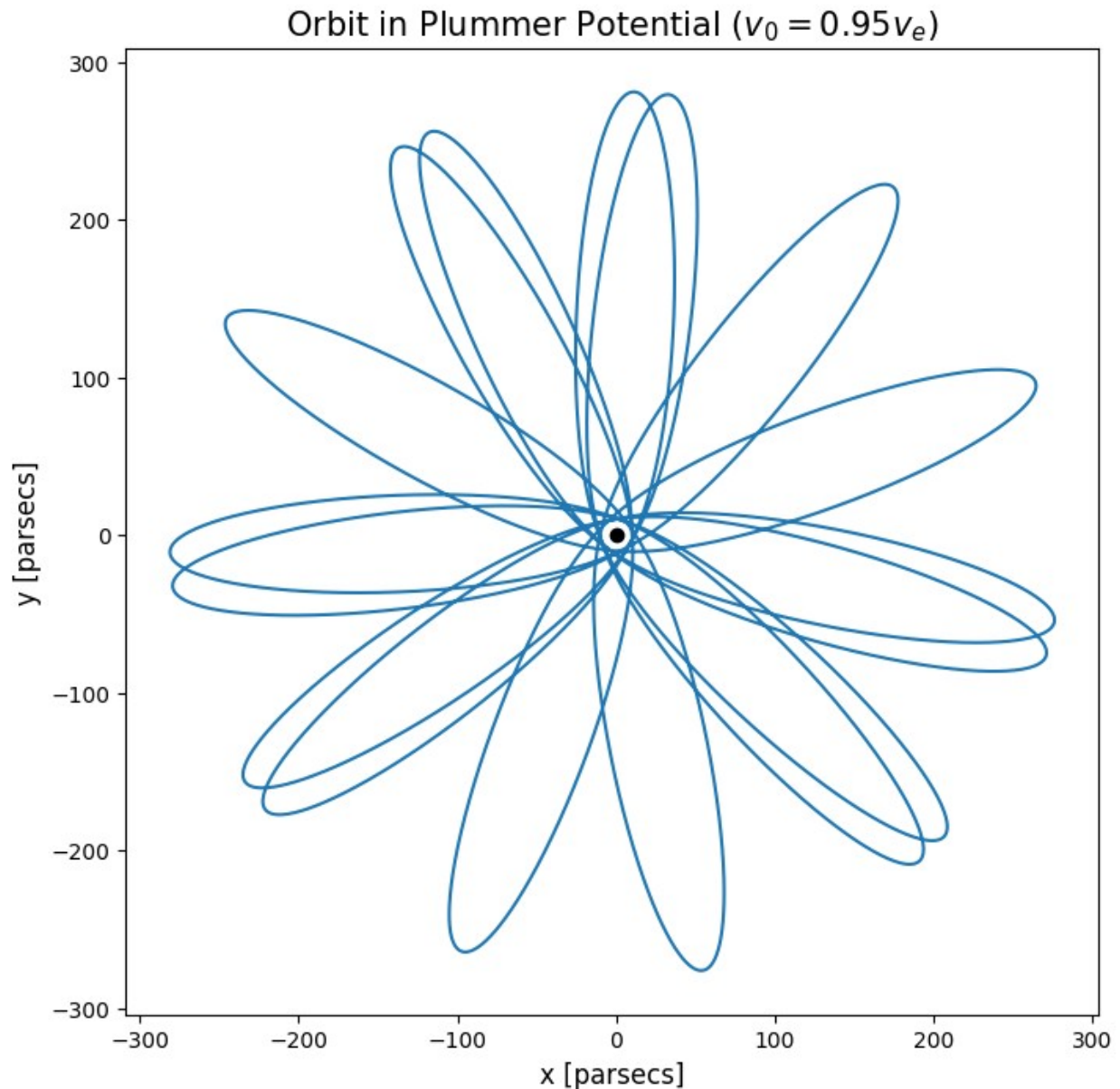
Pregunta 1(b)

Primero para sacar la velocidad de escape, recordamos que esta viene dada por $v_e = \sqrt{2|\phi(r)|}$, donde ϕ corresponde al potencial considerado. Para el caso actual consideramos potencial de plummer para el cual podemos reemplazar los valores numericos que conocemos para Omega Centauri.

```
v_esc =  
np.sqrt(2*c.G*mass*u.M_sun/(np.sqrt((b*u.pc)**2+(10*u.pc)**2)))  
  
v_esc.to(u.km/u.s)  
<Quantity 35.36369396 km / s>
```

Convertimos en las unidades que definen el sistema

```
v_esc.to(u.pc/u.Myr)  
<Quantity 36.16688002 pc / Myr>  
  
unidades = UnitSystem(u.pc, u.Myr, u.radian, u.Msun)  
  
mass = 4.05e6  
b = 26  
  
plummer = gp.PlummerPotential(m=mass, b=b, units=unidades)  
  
r_init = 10  
v_circ = plummer.circular_velocity([r_init, 0, 0]).value  
w0 = gd.PhaseSpacePosition(pos=[r_init, 0, 0], vel=[0, 0.95*36.16688,  
0])  
  
orbital_period = 2 * np.pi * r_init / v_circ  
  
orbit = plummer.integrate_orbit(w0, dt=0.001, n_steps=int(200 *  
orbital_period / 0.001))  
  
fig, ax = plt.subplots(1, 1, figsize=(8, 8))  
ax.scatter(0, 0, color='black', marker='o')  
ax.plot(orbit.x, orbit.y)  
ax.set_xlabel('x [parsecs]', size=12)  
ax.set_ylabel('y [parsecs]', size=12)  
ax.set_title('Orbit in Plummer Potential ($v_0=0.95v_e$)', size=15)  
plt.show()
```



Podemos ver que para un periodo largo de tiempo la orbita sigue siendo cerrada.

Pregunta 1(c)

Considerarmos diferencias de 1 orden de magnitud para los valores; 0.2, 20 y 2000 pc y para cada uno tambien se consideran las velocidades

```
unidades = UnitSystem(u.pc, u.Myr, u.radian, u.Msun)
mass = 4.05e6
```

```

b = 26

potential = gp.PlummerPotential(m=mass, b=b, units=unidades)

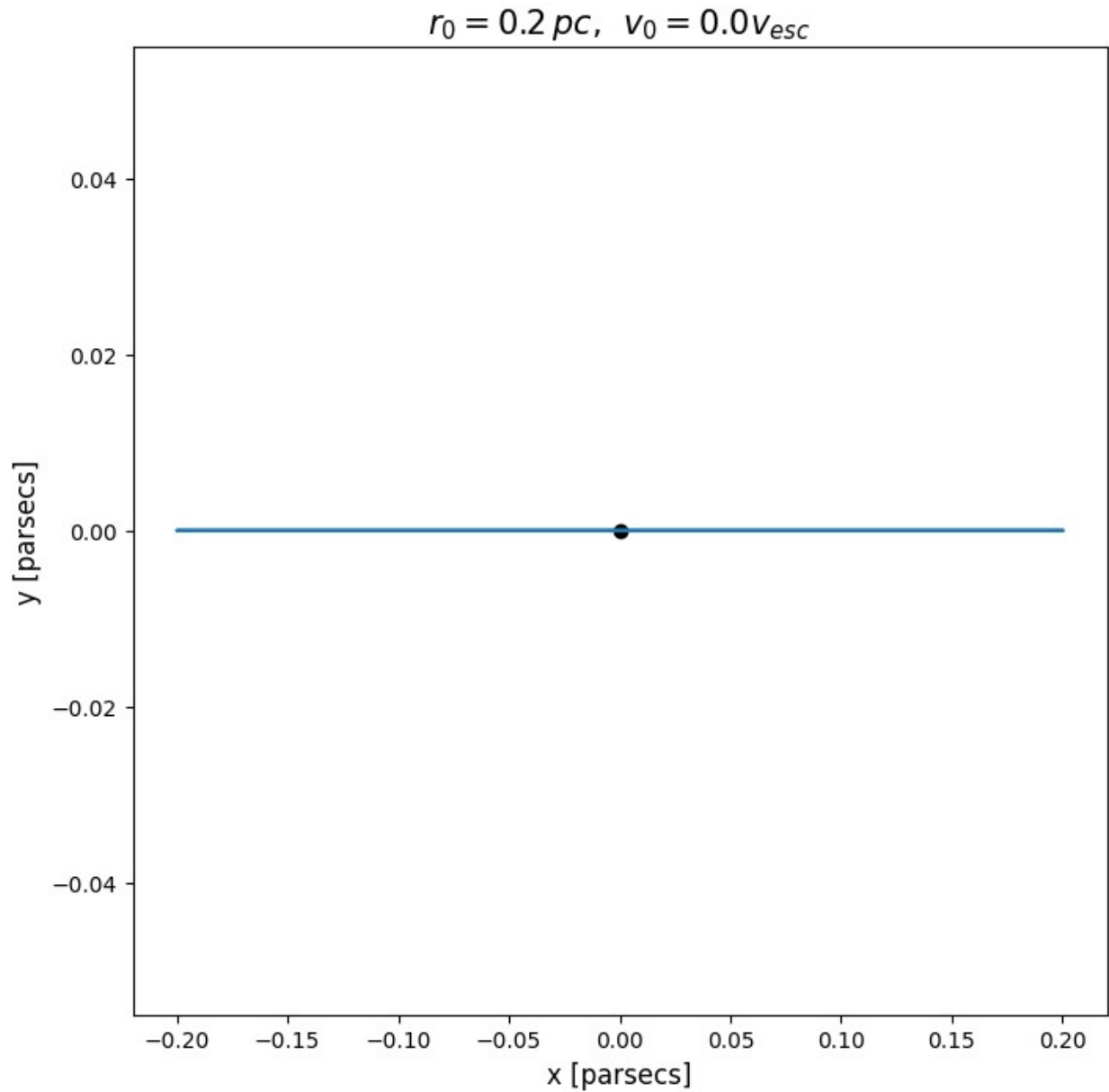
r_init = 0.2
v_circ = potential.circular_velocity([r_init, 0, 0]).value
w0 = gd.PhaseSpacePosition(pos=[r_init, 0, 0], vel=[0, 0, 0])

orbital_period = 2 * np.pi * r_init / v_circ

orbit = potential.integrate_orbit(w0, dt=0.001, n_steps=int(100 *
orbital_period / 0.001))

fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(0, 0, color='black', marker='o')
ax.plot(orbit.x, orbit.y)
ax.set_xlabel('x [parsecs]', size=12)
ax.set_ylabel('y [parsecs]', size=12)
ax.set_title('$r_0=0.2\,pc$, $v_0=0.0\,v_{esc}$', size=15)
plt.show()

```



```
unidades = UnitSystem(u.pc, u.Myr, u.radian, u.Msun)

mass = 4.05e6
b = 26

potential = gp.PlummerPotential(m=mass, b=b, units=unidades)

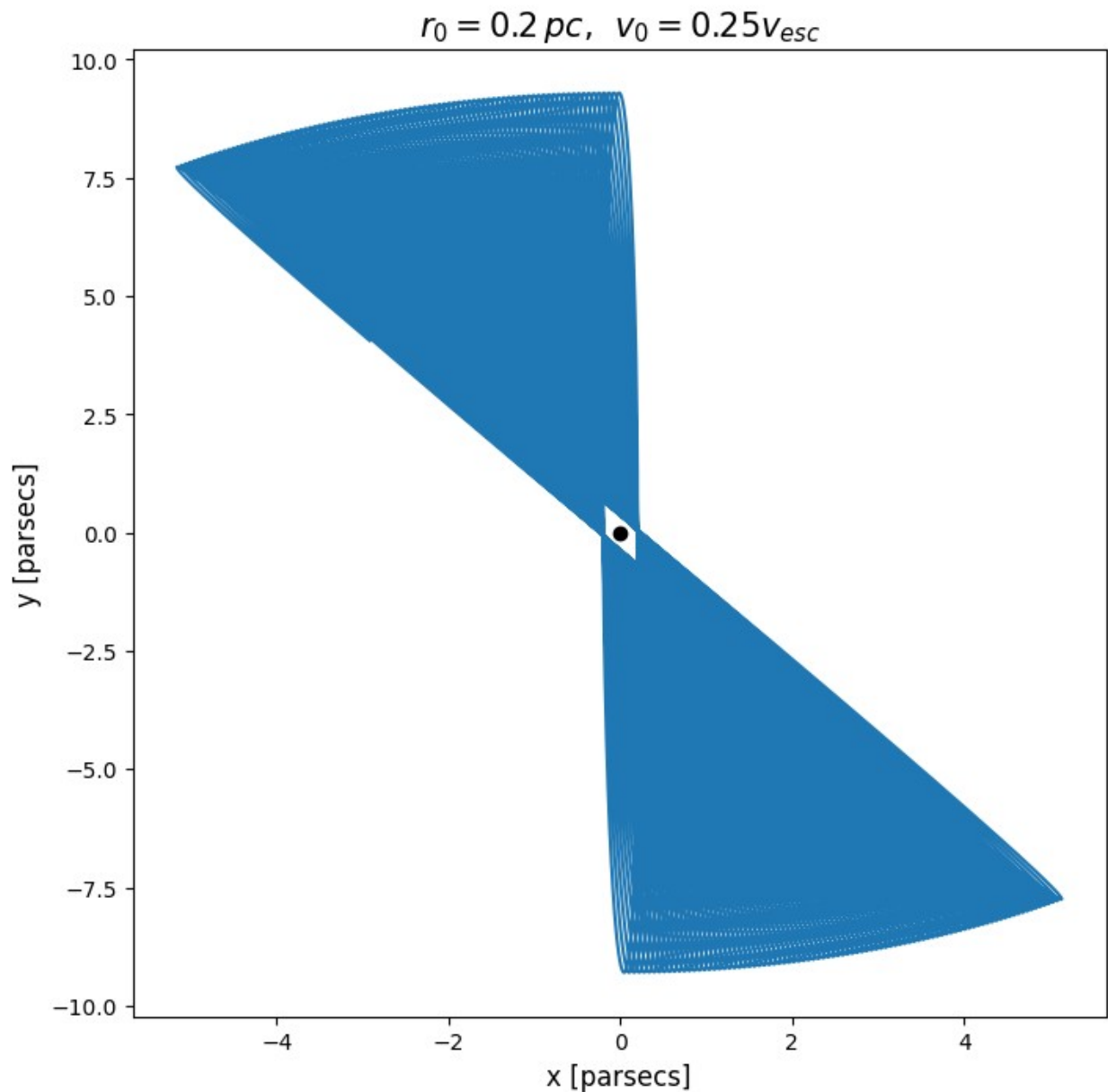
r_init = 0.2
v_circ = potential.circular_velocity([r_init, 0, 0]).value
w0 = gd.PhaseSpacePosition(pos=[r_init, 0, 0], vel=[0, 0.25*36.16688, 0])

orbital_period = 2 * np.pi * r_init / v_circ
```



```
orbit = potential.integrate_orbit(w0, dt=0.001, n_steps=int(100 *
orbital_period / 0.001))
```

```
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(0, 0, color='black', marker='o')
ax.plot(orbit.x, orbit.y)
ax.set_xlabel('x [parsecs]', size=12)
ax.set_ylabel('y [parsecs]', size=12)
ax.set_title('$r_0=0.2\,pc$, $v_0=0.25\,v_{esc}$', size=15)
plt.show()
```



```

unidades = UnitSystem(u.pc, u.Myr, u.radian, u.Msun)

mass = 4.05e6
b = 26

potential = gp.PlummerPotential(m=mass, b=b, units=unidades)

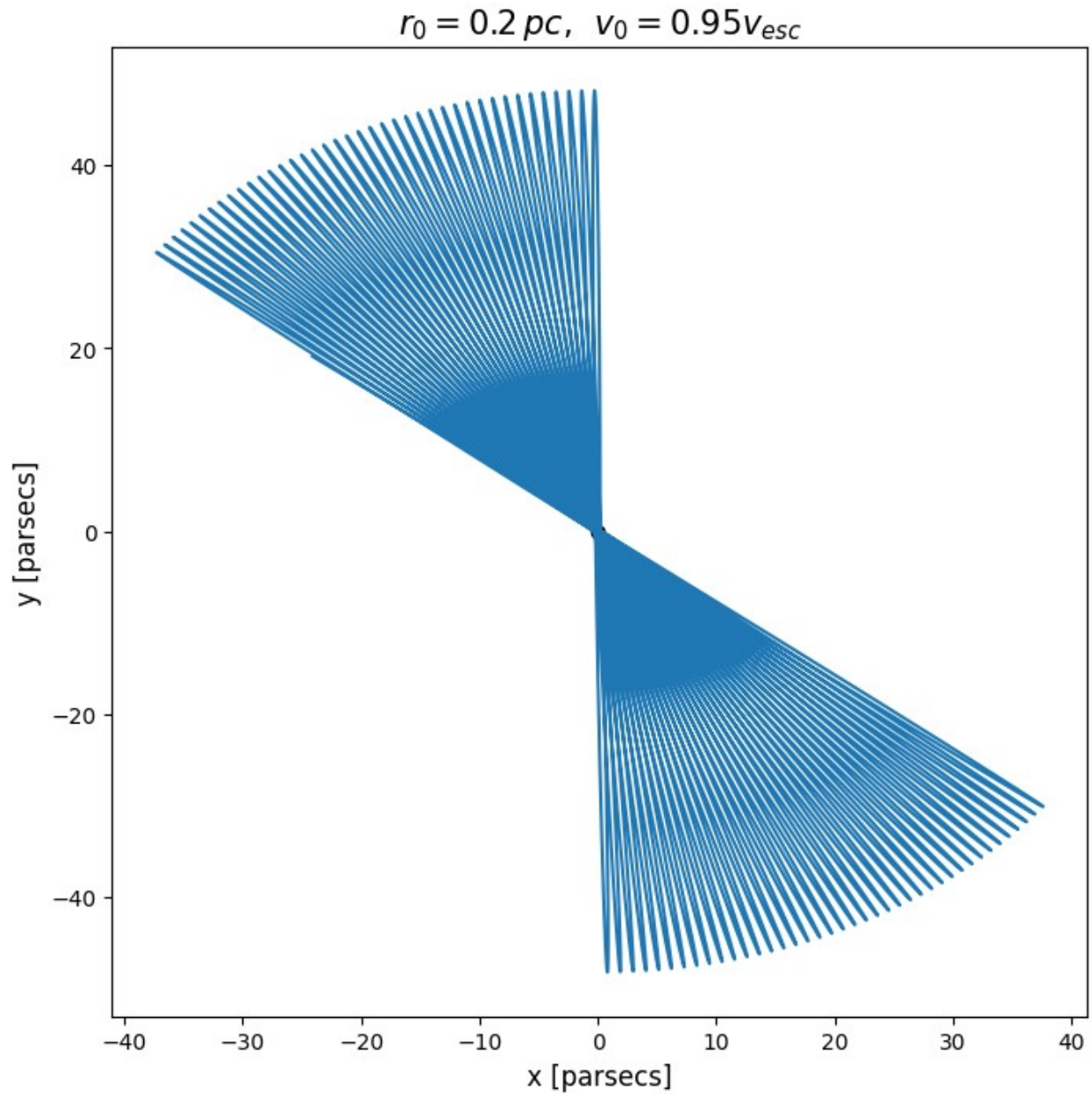
r_init = 0.2
v_circ = potential.circular_velocity([r_init, 0, 0]).value
w0 = gd.PhaseSpacePosition(pos=[r_init, 0, 0], vel=[0, 0.75*36.16688,
0])

orbital_period = 2 * np.pi * r_init / v_circ

orbit = potential.integrate_orbit(w0, dt=0.001, n_steps=int(100 *
orbital_period / 0.001))

fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(0, 0, color='black', marker='o')
ax.plot(orbit.x, orbit.y)
ax.set_xlabel('x [parsecs]', size=12)
ax.set_ylabel('y [parsecs]', size=12)
ax.set_title('$r_0=0.2\backslash,pc$, $v_0=0.95 v_{\text{esc}}$', size=15)
plt.show()

```



```

unidades = UnitSystem(u.pc, u.Myr, u.radian, u.Msun)

mass = 4.05e6
b = 26

potential = gp.PlummerPotential(m=mass, b=b, units=unidades)

r_init = 20
v_circ = potential.circular_velocity([r_init, 0, 0]).value
w0 = gd.PhaseSpacePosition(pos=[r_init, 0, 0], vel=[0, 0.0*36.16688,
0])

```

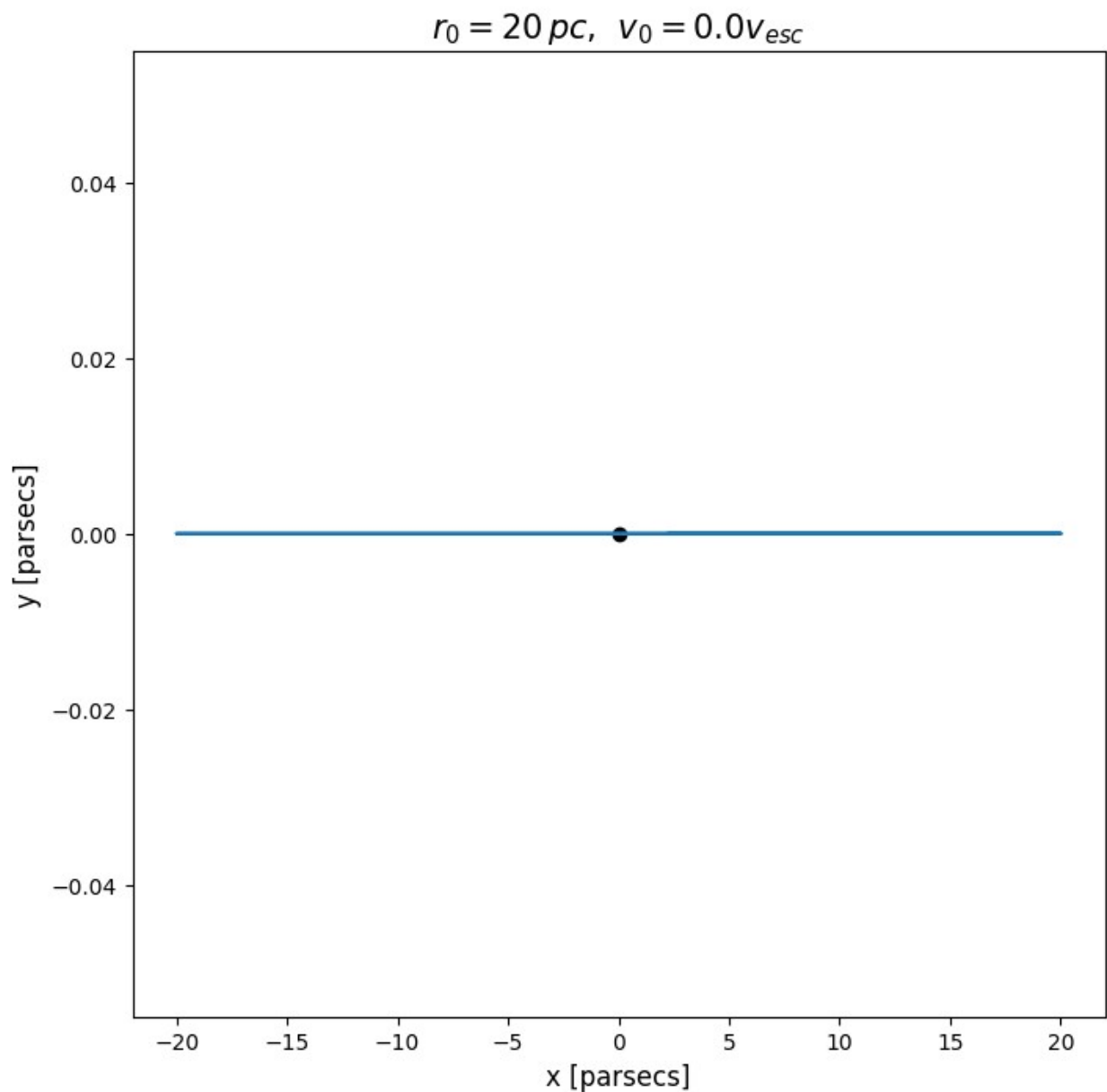
```

orbital_period = 2 * np.pi * r_init / v_circ

orbit = potential.integrate_orbit(w0, dt=0.001, n_steps=int(100 *
orbital_period / 0.001))

fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(0, 0, color='black', marker='o')
ax.plot(orbit.x, orbit.y)
ax.set_xlabel('x [parsecs]', size=12)
ax.set_ylabel('y [parsecs]', size=12)
ax.set_title('$r_0=20\,pc$, $v_0=0.0\,v_{esc}$', size=15)
plt.show()

```



```

unidades = UnitSystem(u.pc, u.Myr, u.radian, u.Msun)

mass = 4.05e6
b = 26

potential = gp.PlummerPotential(m=mass, b=b, units=unidades)

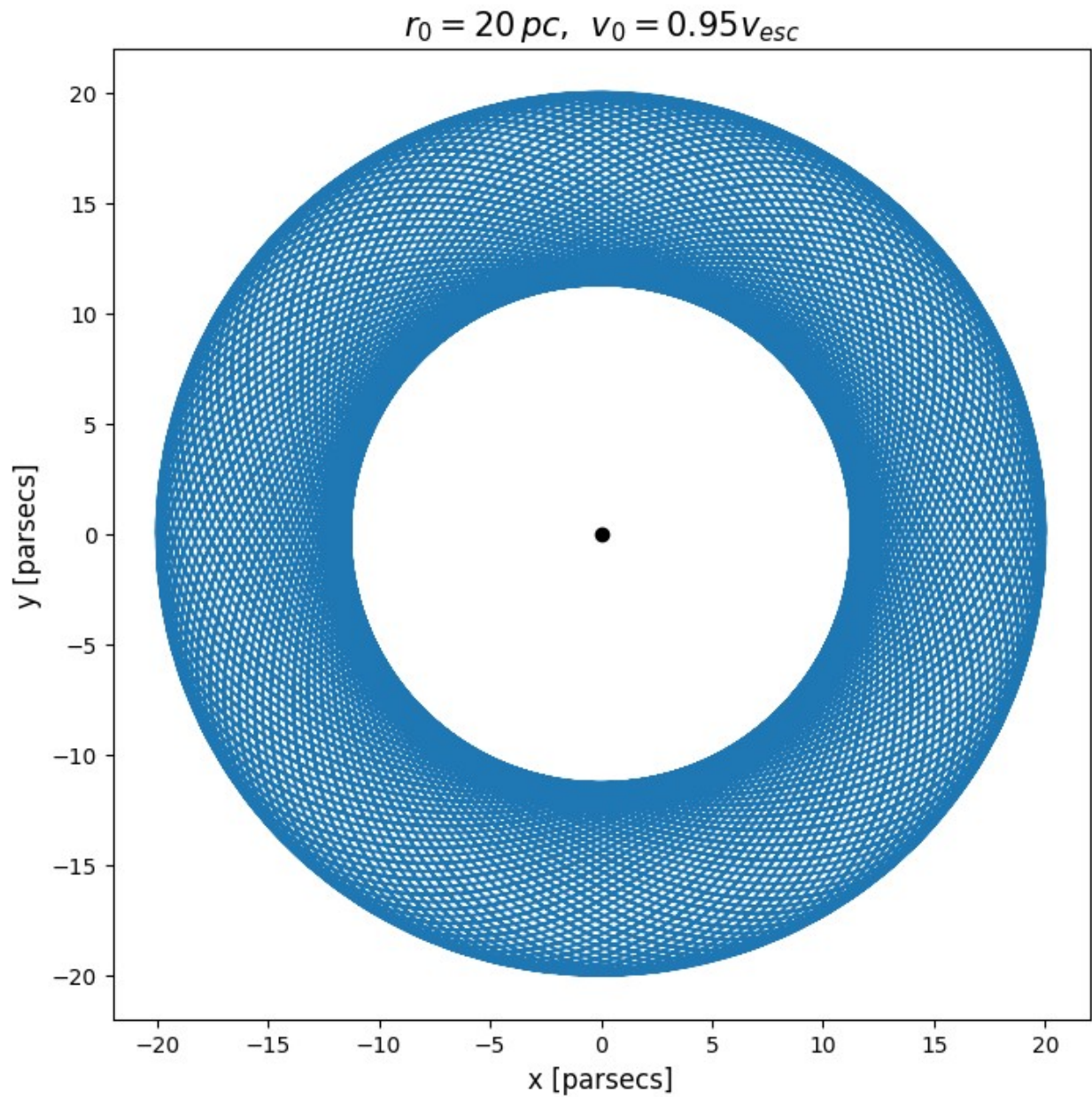
r_init = 20
v_circ = potential.circular_velocity([r_init, 0, 0]).value
w0 = gd.PhaseSpacePosition(pos=[r_init, 0, 0], vel=[0, 0.25*36.16688,
0])

orbital_period = 2 * np.pi * r_init / v_circ

orbit = potential.integrate_orbit(w0, dt=0.001, n_steps=int(100 *
orbital_period / 0.001))

fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(0, 0, color='black', marker='o')
ax.plot(orbit.x, orbit.y)
ax.set_xlabel('x [parsecs]', size=12)
ax.set_ylabel('y [parsecs]', size=12)
ax.set_title('$r_0=20\backslash,pc$, $v_0=0.25 v_{esc}$', size=15)
plt.show()

```



```

unidades = UnitSystem(u.pc, u.Myr, u.radian, u.Msun)

mass = 4.05e6
b = 26

potential = gp.PlummerPotential(m=mass, b=b, units=unidades)

r_init = 20
v_circ = potential.circular_velocity([r_init, 0, 0]).value
w0 = gd.PhaseSpacePosition(pos=[r_init, 0, 0], vel=[0, 0.75*36.16688,
0])

```

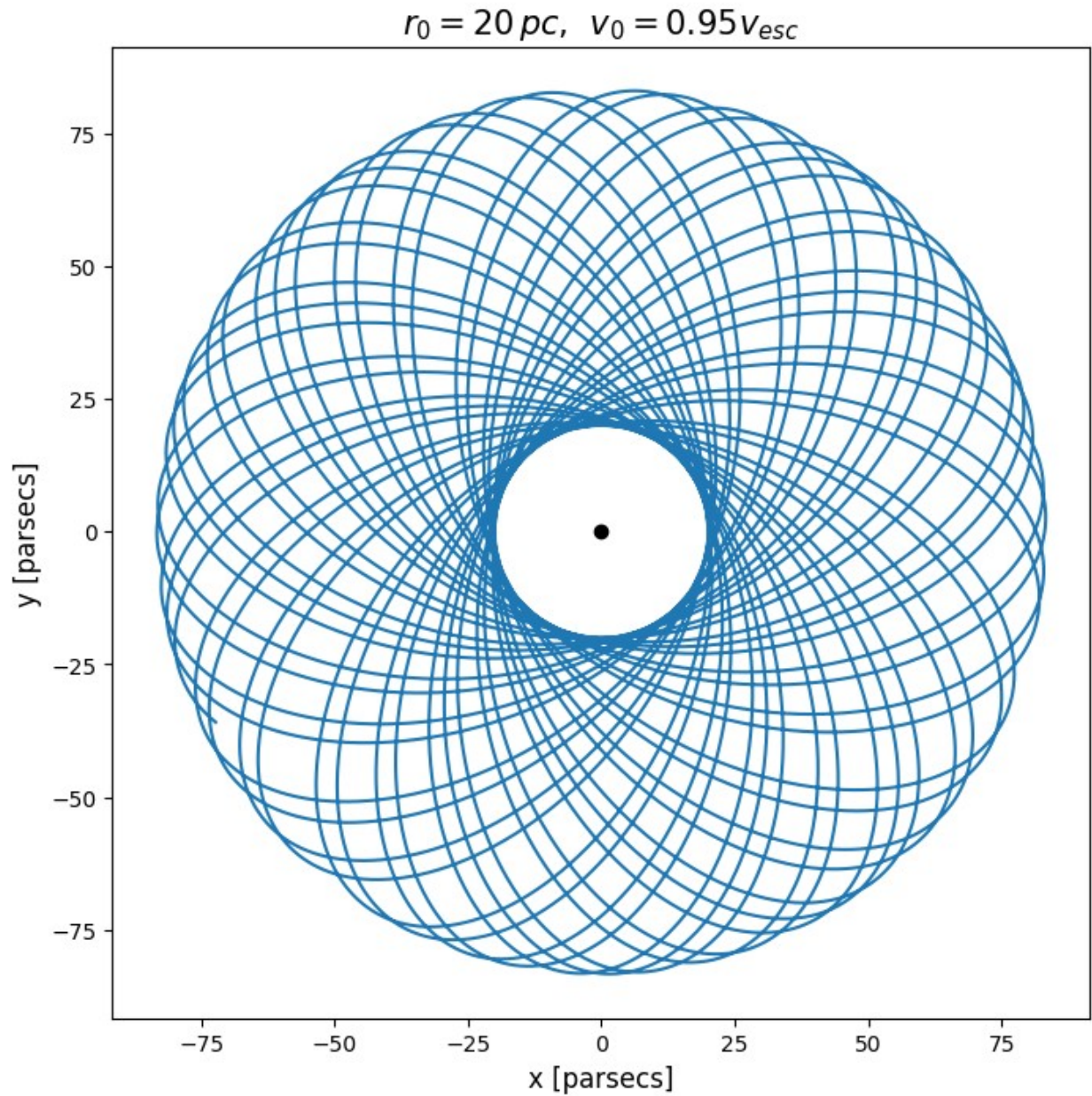
```

orbital_period = 2 * np.pi * r_init / v_circ

orbit = potential.integrate_orbit(w0, dt=0.001, n_steps=int(100 *
orbital_period / 0.001))

fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(0, 0, color='black', marker='o')
ax.plot(orbit.x, orbit.y)
ax.set_xlabel('x [parsecs]', size=12)
ax.set_ylabel('y [parsecs]', size=12)
ax.set_title('$r_0=20\backslash,pc$, $v_0=0.95 v_{\text{esc}}$', size=15)
plt.show()

```

```

unidades = UnitSystem(u.pc, u.Myr, u.radian, u.Msun)

mass = 4.05e6
b = 26

plummer = gp.PlummerPotential(m=mass, b=b, units=unidades)

r_init = 200
v_circ = plummer.circular_velocity([r_init, 0, 0]).value
w0 = gd.PhaseSpacePosition(pos=[r_init, 0, 0], vel=[0, 0*36.16688, 0])

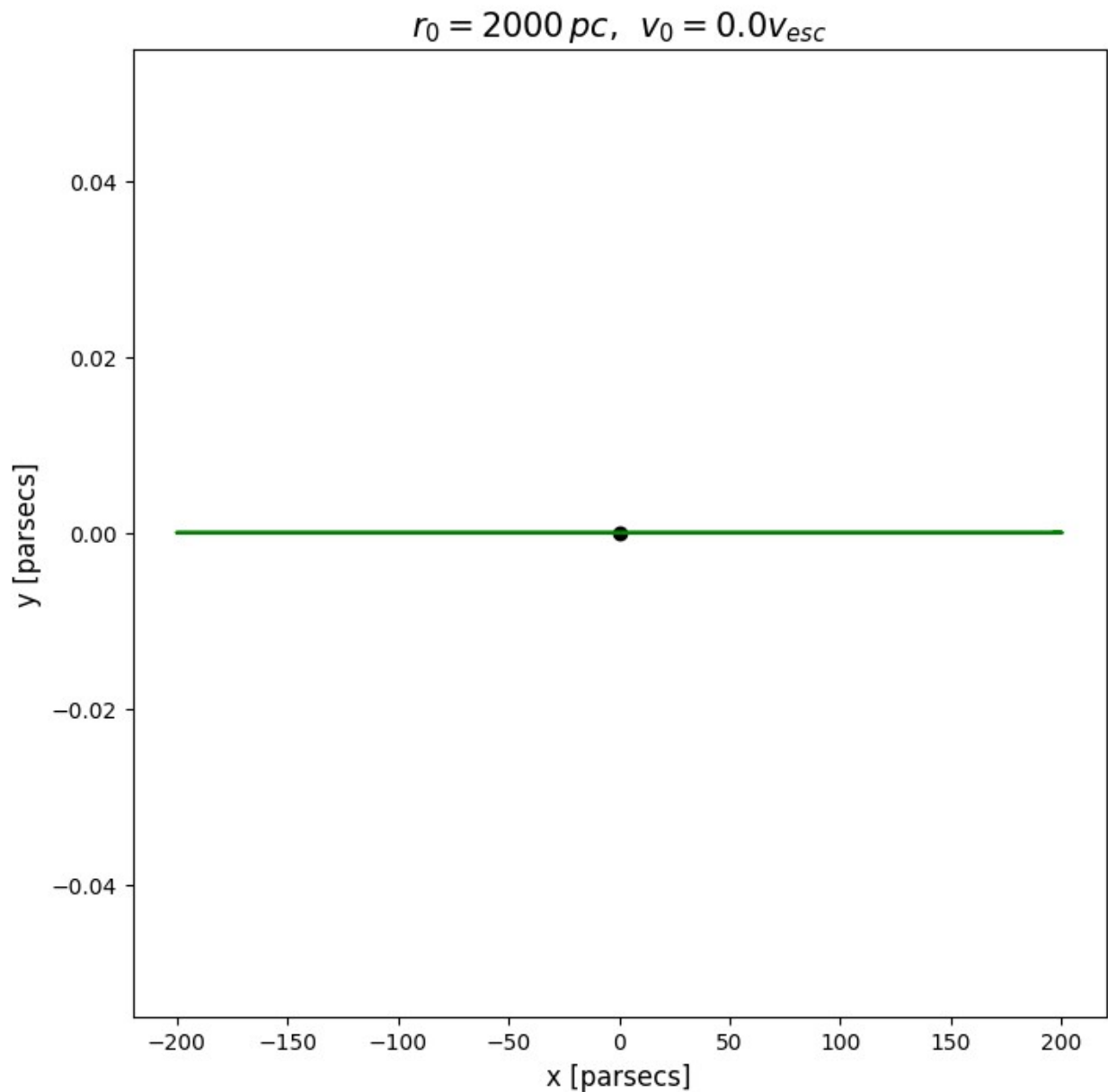
orbital_period = 2 * np.pi * r_init / v_circ

```



```
orbit = plummer.integrate_orbit(w0, dt=0.001, n_steps=int(100 *
orbital_period / 0.001))
```

```
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(0, 0, color='black', marker='o')
ax.plot(orbit.x, orbit.y, color='green')
ax.set_xlabel('x [parsecs]', size=12)
ax.set_ylabel('y [parsecs]', size=12)
ax.set_title('$r_0=2000\backslash,pc$, $v_0=0.0 v_{esc}$', size=15)
plt.show()
```



```

unidades = UnitSystem(u.pc, u.Myr, u.radian, u.Msun)

mass = 4.05e6
b = 26

plummer = gp.PlummerPotential(m=mass, b=b, units=unidades)

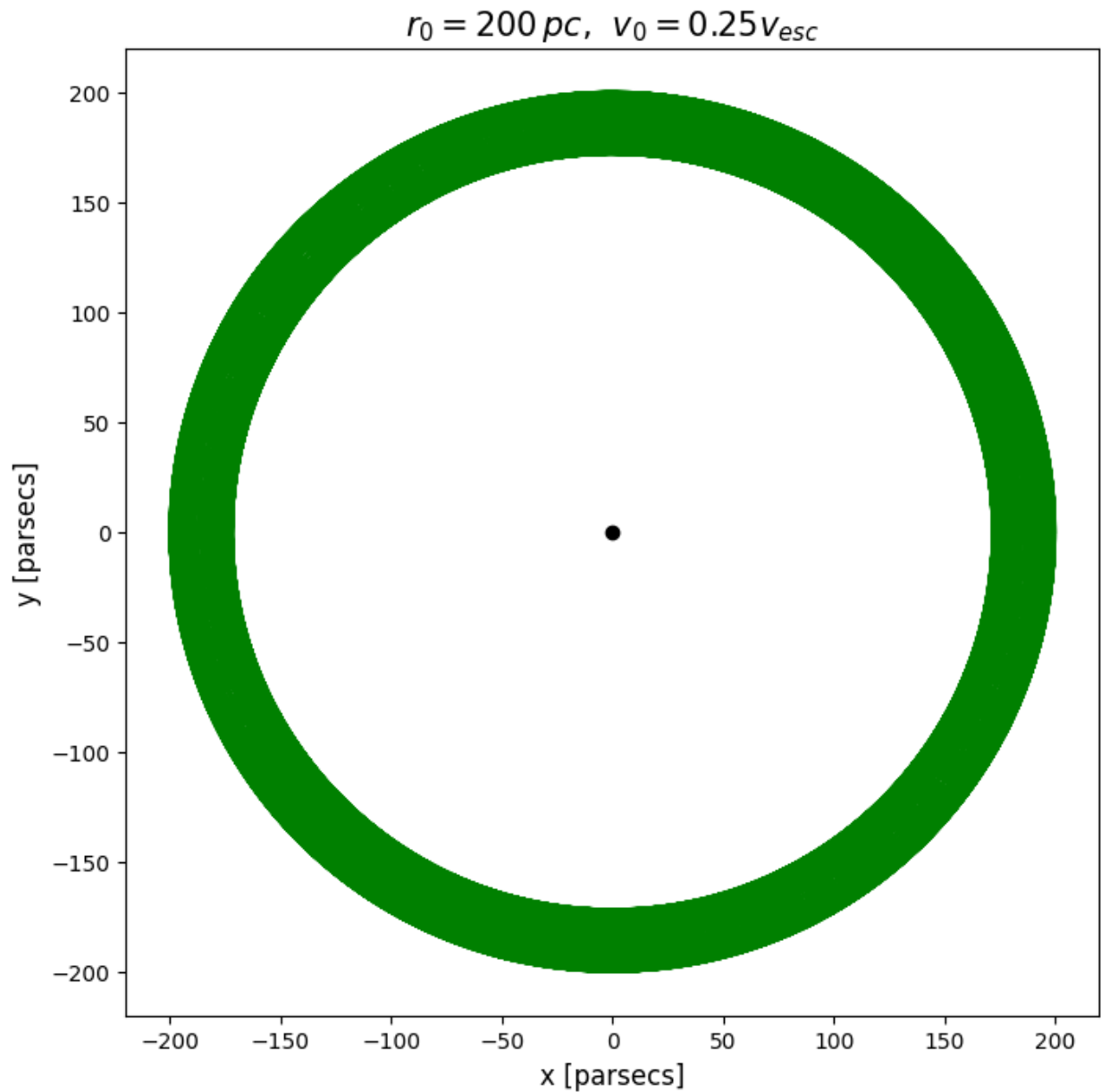
r_init = 200
v_circ = plummer.circular_velocity([r_init, 0, 0]).value
w0 = gd.PhaseSpacePosition(pos=[r_init, 0, 0], vel=[0, 0.25*36.16688,
0])

orbital_period = 2 * np.pi * r_init / v_circ

orbit = plummer.integrate_orbit(w0, dt=0.001, n_steps=int(100 *
orbital_period / 0.001))

fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(0, 0, color='black', marker='o')
ax.plot(orbit.x, orbit.y, color='green')
ax.set_xlabel('x [parsecs]', size=12)
ax.set_ylabel('y [parsecs]', size=12)
ax.set_title('$r_0=200\backslash,pc$, $v_0=0.25 v_{esc}$', size=15)
plt.show()

```



```

unidades = UnitSystem(u.pc, u.Myr, u.radian, u.Msun)

mass = 4.05e6
b = 26

plummer = gp.PlummerPotential(m=mass, b=b, units=unidades)

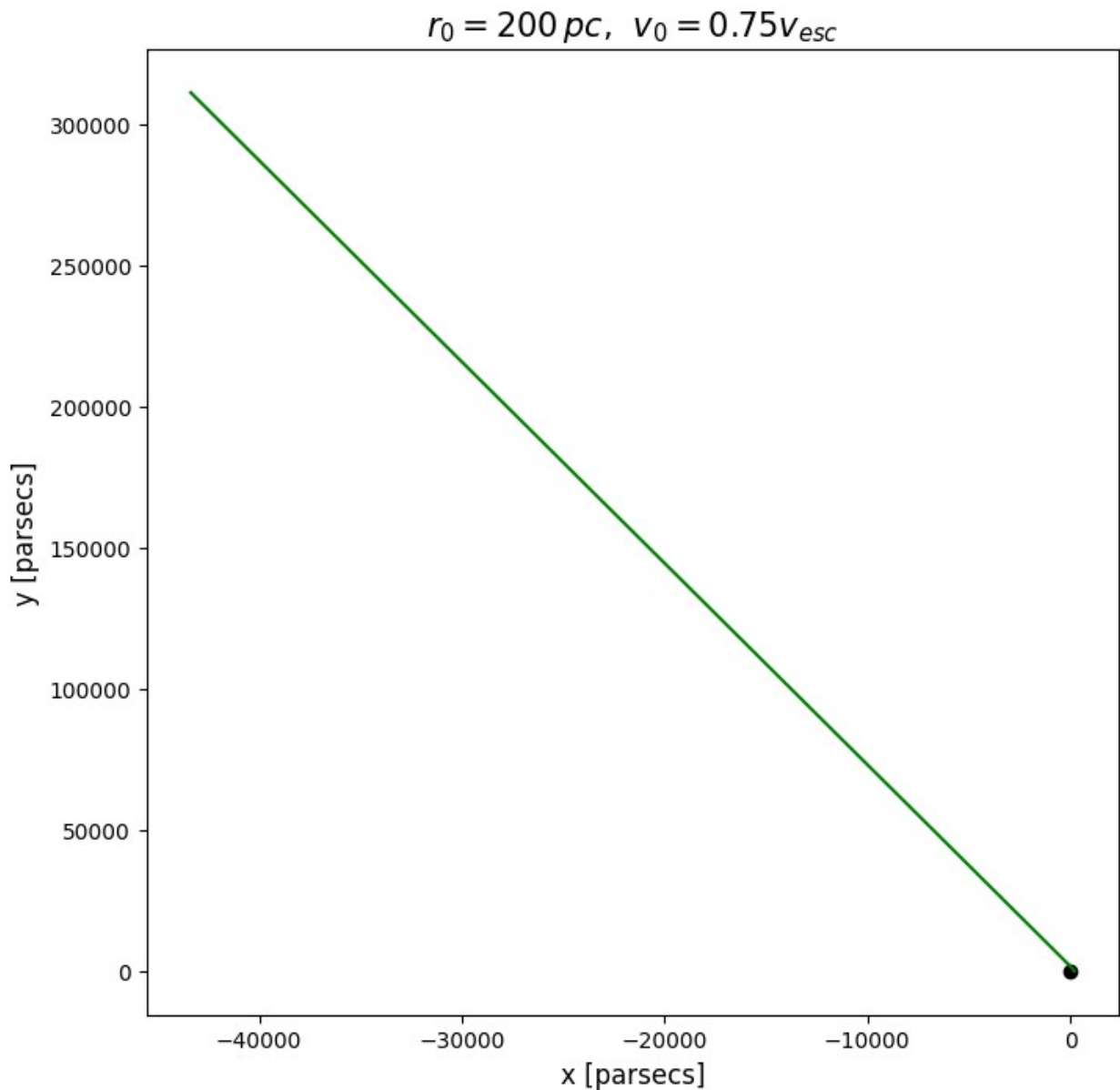
r_init = 200
v_circ = plummer.circular_velocity([r_init, 0, 0]).value
w0 = gd.PhaseSpacePosition(pos=[r_init, 0, 0], vel=[0, 0.75*36.16688, 0])

orbital_period = 2 * np.pi * r_init / v_circ

```

```
orbit = plummer.integrate_orbit(w0, dt=0.001, n_steps=int(100 *
orbital_period / 0.001))
```

```
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(0, 0, color='black', marker='o')
ax.plot(orbit.x, orbit.y, color='green')
ax.set_xlabel('x [parsecs]', size=12)
ax.set_ylabel('y [parsecs]', size=12)
ax.set_title('$r_0=200\backslash pc$, $v_0=0.75 v_{esc}$', size=15)
plt.show()
```



Se observa que para las orbitas con $v_0 = 0$, el sistema se comporta como un oscilador armónico en 1D. Además podemos ver que para las velocidades iniciales mas cercanas a la velocidad circular, las orbitas toman valores de excentricidad menores.

Pregunta 1(d) - HECHA A MANO

##Pregunta 2(c)

```
v = 5* u.km/u.s
v_cunits = v.to(u.pc/u.Myr)
unidades = UnitSystem(u.pc, u.Myr, u.radian, u.Msun)

mass = 6.8e10
a = 3000
b = 280
R = 8000
z = 100

plummer = gp.MiyamotoNagaiPotential(m=mass, a=a, b=b,units=unidades)

r_init = R
v_circ = plummer.circular_velocity([r_init, 0, z]).value
w0 = gd.PhaseSpacePosition(pos=[r_init, 0, z], vel=[v_cunits.value,
v_circ, 0])

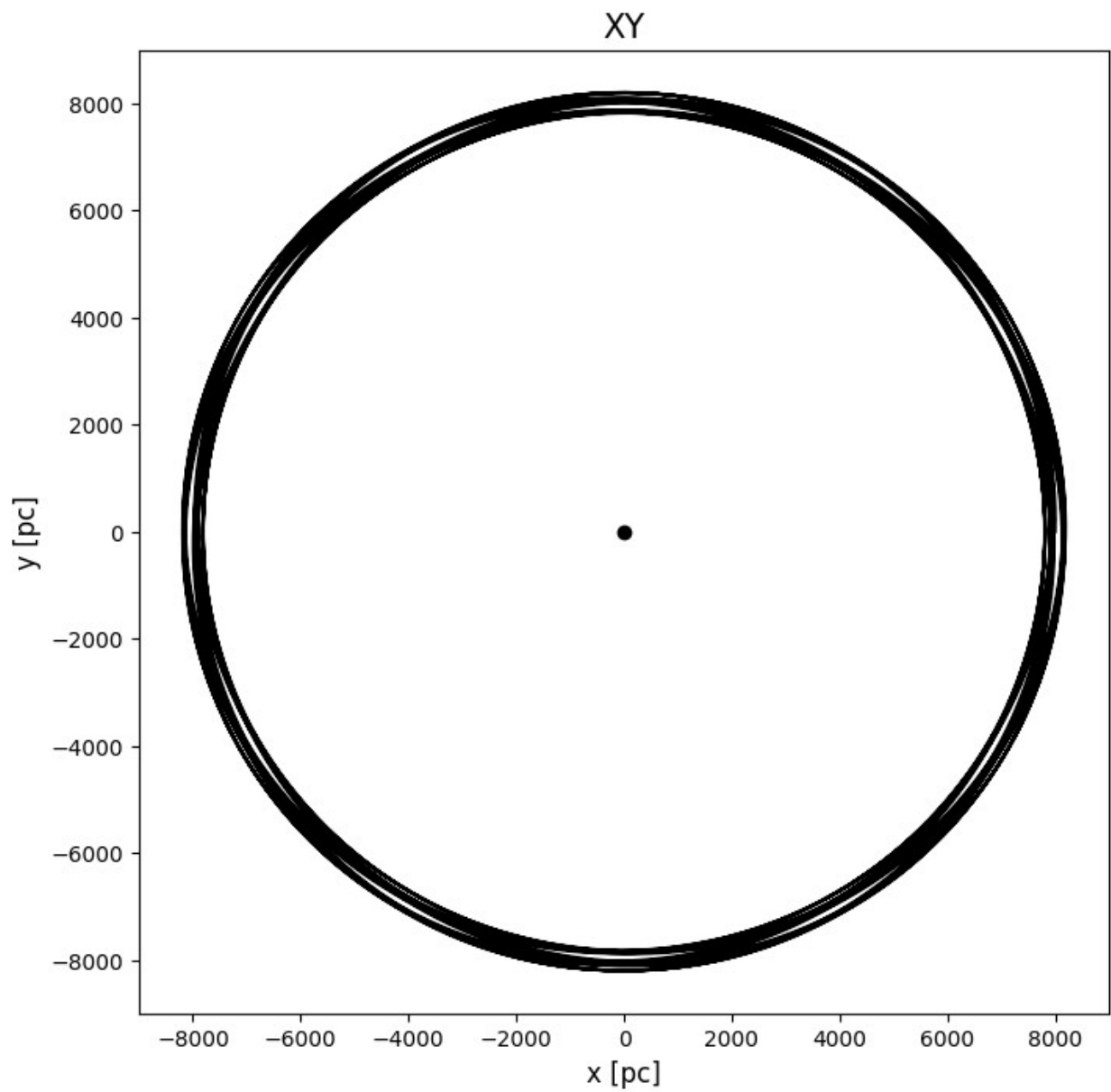
orbital_period = 2 * np.pi * r_init / v_circ
orbit = plummer.integrate_orbit(w0, dt=0.01, n_steps=int(25 *
orbital_period / 0.01))

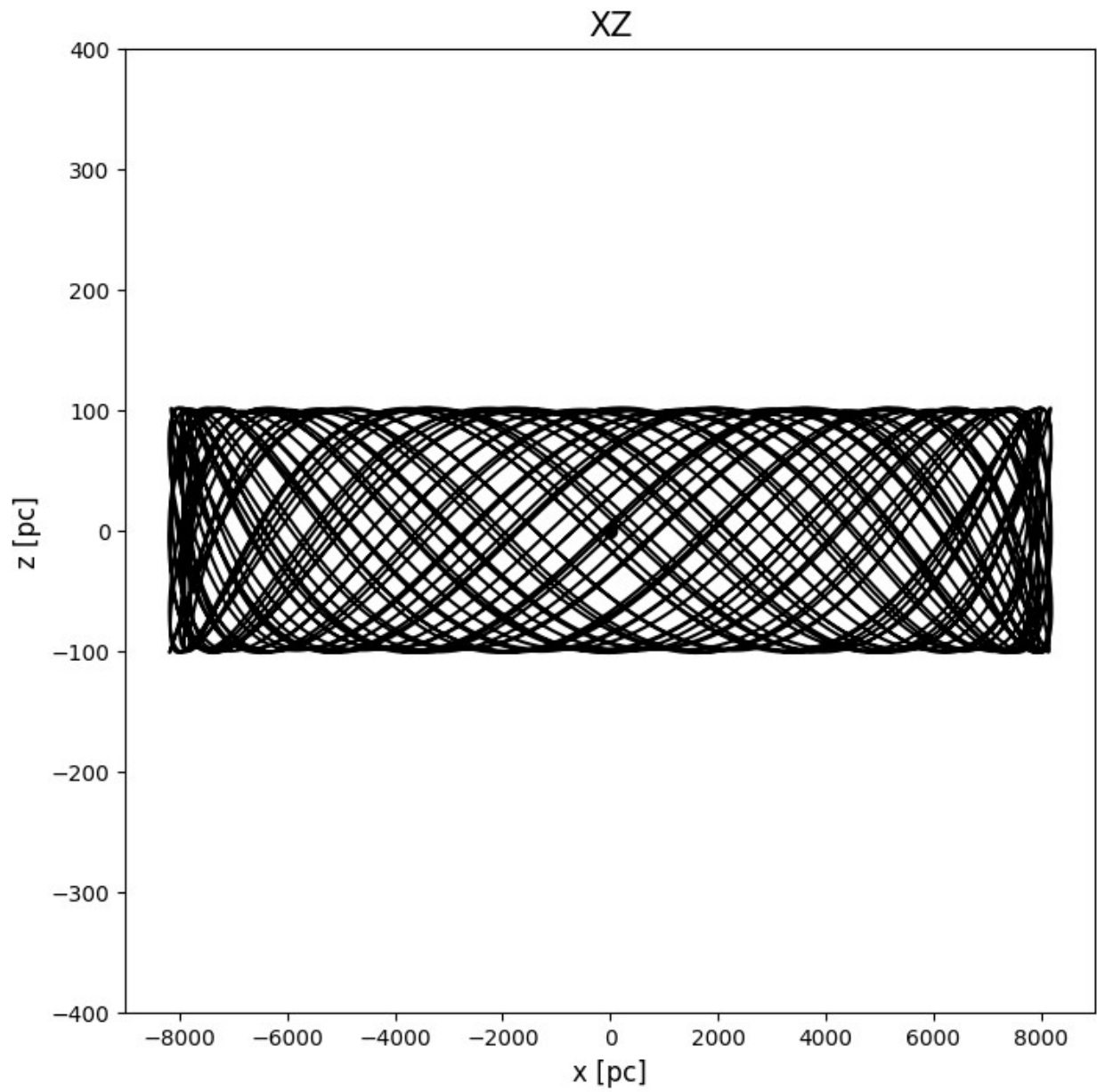
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(0, 0, color='black', marker='o')
ax.plot(orbit.x, orbit.y, color='black')
ax.set_xlim([-9000,9000])
ax.set_ylim([-9000,9000])
ax.set_xlabel('x [pc]', size=12)
ax.set_ylabel('y [pc]', size=12)
ax.set_title('XY', size=15)

fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(0, 0, color='black', marker='o')
ax.plot(orbit.x, orbit.z, color='black')
ax.set_xlim([-9000,9000])
```

```
ax.set_ylim([-400,400])
ax.set_xlabel('x [pc]', size=12)
ax.set_ylabel('z [pc]', size=12)
ax.set_title('XZ', size=15)

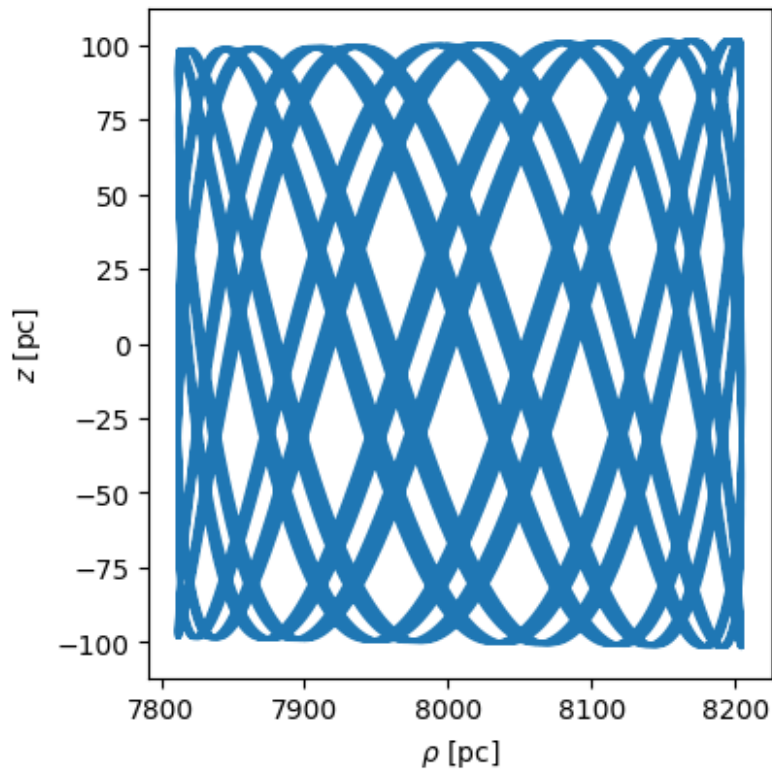
plt.show()
```





En coordenadas cilíndricas:

```
fig = orbit.represent_as('cylindrical').plot(['rho', 'z'])
```



Ahora para comparar, convertimos las unidades de los valores obtenidos en el desarrollo

```

nu = 2.36*10**(-15)*(1/u.s)
kappa = 8.25*10**(-16)*(1/u.s)

nu_myr = nu.to(1/u.Myr)
kappa_myr = kappa.to(1/u.Myr)

nu_myr, kappa_myr

(<Quantity 0.07447594 1 / Myr>, <Quantity 0.02603502 1 / Myr>)

x_0=200
z_0=100
kappa_=0.02603
nu=0.07447

#Definimos las funciones dadas por el enunciado

def x(T):
    return x0*np.sin(kappa_myr.value*T)

def z(T):
    return z0*np.cos(nu_myr.value*T)

```



```

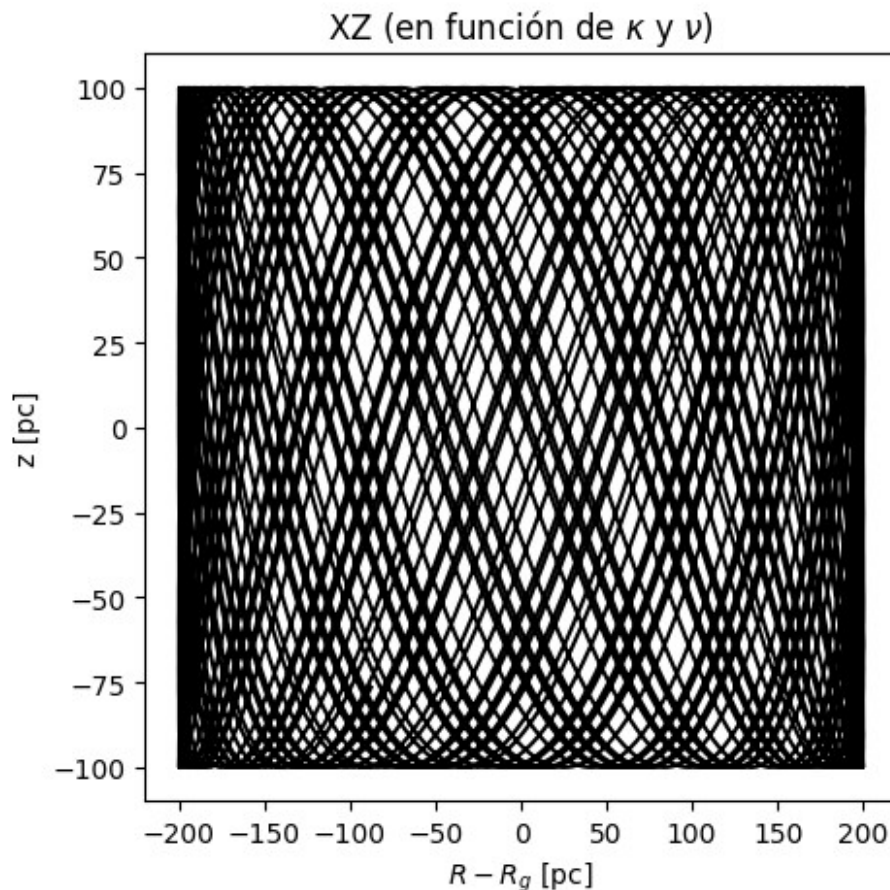
t_start = 0
t_end = 25*orbital_period
dt = 0.01

t = np.arange(t_start, t_end, dt)

x = x(t)
z = z(t)

plt.figure(figsize=(5, 5))
plt.plot(x, z, c='black')
plt.xlabel('$R-R_g$ [pc]')
plt.ylabel('$z$ [pc]')
plt.title('XZ (en función de  $\kappa$  y  $\nu$ )')
#plt.grid(True)
plt.show()

```



Vemos que ambos graficos describen orbitas casi identicas

